Фамилия, имя: _____ Группа: _____

Материал: лекции 1, 2, 3, 5 и 6. Импорты и внешние библиотеки не нужны.

Работа выполняется на бумаге, без запуска кода. В каждой задаче напишите функцию на Python. Читать данные через `input` и печатать результат не нужно: функция принимает аргументы и возвращает результат. Аннотации типов необязательны. Можно использовать встроенные функции и методы Python. Исходные списки менять нельзя.

Задачи независимы, их можно решать в любом порядке. Письменные объяснения и ручная трассировка не требуются. За частично верное решение начисляются баллы.

Задача 1. Оставить цифры заданной чётности

2 балла

Напишите функцию `keep_digit_parity(number, parity)`.

`number` - целое неотрицательное число. `parity` равен `0` или `1`. При `parity == 0` оставьте только чётные цифры, при `parity == 1` - только нечётные.

Порядок оставшихся цифр должен совпадать с их порядком в исходном числе. Верните результат как целое число: начальные нули в нём не сохраняются. Если ни одной цифры не осталось, верните `0`. Число `0` считайте записью из одной цифры `0`.

Преобразовывать число или отдельные цифры в строки нельзя.

```
def keep_digit_parity(number: int, parity: int) -> int:
    ...
```

Примеры:

```
assert keep_digit_parity(58216, 0) == 826
assert keep_digit_parity(58216, 1) == 51
assert keep_digit_parity(135, 0) == 0
assert keep_digit_parity(1002, 0) == 2
assert keep_digit_parity(0, 0) == 0
```

Задача 2. Соседние слова разной длины

2 балла

Напишите функцию `different_length_neighbors(text)`.

Верните список пар соседних слов, для которых длины слов различаются. Каждая пара должна быть кортежем `(первое слово, второе слово)` в порядке их появления в строке.

Проверяются все соседние пары исходной строки. Одно слово может входить в две пары. Сохраните повторы и исходный регистр букв.

Строка содержит только буквы и пробелы. Пробелов между словами, в начале и конце строки может быть несколько. Если подходящих пар нет или слов меньше двух, верните пустой список.

```
def different_length_neighbors(text: str) -> list[tuple[str, str]]:
    ...
```

Примеры:

```
assert different_length_neighbors('кот кит и мы ты') == [('кит', 'и'), ('и', 'мы')]
assert different_length_neighbors('я мы кот') == [('я', 'мы'), ('мы', 'кот')]
assert different_length_neighbors('кот кит') == []
assert different_length_neighbors('  Кот   Кит  ') == []
assert different_length_neighbors('Привет') == []
assert different_length_neighbors('   ') == []
assert different_length_neighbors('') == []
```

Задача 3. Первый блок отрицательных чисел

2,5 балла

Напишите функцию `first_negative_run(values, k)`.

`values` - список целых чисел, `k` - целое неотрицательное число.

Верните новый список из первого непрерывного блока длины `k`, в котором все числа строго отрицательные. Если таких блоков несколько, выберите тот, который начинается раньше. Элементы блока должны идти подряд в исходном списке: пропускать неподходящие элементы внутри блока нельзя.

Если такого блока нет, верните пустой список. При `k == 0` также верните пустой список и не просматривайте элементы. Как только первый подходящий блок найден, просмотр исходного списка должен прекратиться.

```
def first_negative_run(values: list[int], k: int) -> list[int]:
    ...
```

Примеры:

```
assert first_negative_run([-1, 0, -3, -5, -7], 2) == [-3, -5]
assert first_negative_run([-1, -3, -5], 2) == [-1, -3]
assert first_negative_run([-1, 2, -3], 2) == []
assert first_negative_run([3, 0, -2], 1) == [-2]
assert first_negative_run([], 2) == []
assert first_negative_run([1, -1, 0], 0) == []
```

Задача 4. Ошибки без игнорируемых кодов

3,5 балла

Напишите функцию `count_errors(errors, ignored)`.

`errors` - список кодов ошибок в журнале, `ignored` - список кодов, которые нужно исключить из отчёта. Коды ошибок - строки.

Верните словарь: ключ - код ошибки, значение - число его вхождений в `errors`, если его нет в `ignored`. Здесь нужно учитывать все повторения в первом списке. Повторы во втором списке ничего не меняют. Значения, встречающиеся только во втором списке, в результат не включайте.

Оба списка содержат строки. Строки сравниваются точно, с учётом регистра; очищать или преобразовывать их не нужно. Порядок ключей не важен. Если после исключения ничего не осталось, верните пустой словарь.

Требование к эффективности: среднее время работы `O(n + m)`, включая подготовку данных, где `n` и `m` - длины двух входных списков. Считайте, что проверка наличия строки в словаре или множестве, добавление записи в словарь и добавление строки в множество в среднем выполняются за `O(1)`. Объяснять сложность письменно не нужно.

```
def count_errors(errors: list[str], ignored: list[str]) -> dict[str, int]:
    ...
```

Примеры:

```
assert count_errors(['E1', 'E2', 'E1', 'E3', 'E2'], ['E2']) == {'E1': 2, 'E3': 1}
assert count_errors(['E1', 'E1'], ['E1', 'E1']) == {}
assert count_errors(['E1', 'E1'], []) == {'E1': 2}
assert count_errors([], ['нет записей']) == {}
```